

TP Classification : Arbre de décision (DT)

Dans ce TP, nous allons prédire si un passager a survécu au naufrage du Titanic en fonction de plusieurs caractéristiques, en utilisant un modèle d'arbre de décision. Le jeu de données `titanic-passengers.csv`, contient les colonnes suivantes :

- `PassengerId` : identifiant du passager.
- `Pclass` : classe du passager (1, 2, 3).
- `Name` : nom du passager.
- `Sex` : sexe du passager (Male ou Female).
- `Age` : âge du passager.
- `SibSp` : nombre de frères et sœurs / époux à bord.
- `Parch` : nombre de parents / enfants à bord.
- `Ticket` : numéro du billet.
- `Fare` : tarif du billet.
- `Cabin` : numéro de la cabine.
- `Embarked` : port d'embarquement (C = Cherbourg; Q = Queenstown; S = Southampton).
- `Survived` : variable cible indiquant si le passager a survécu (1 pour oui, 0 pour non).

1) Chargez le fichier `titanic-passengers.csv`, puis affichez un aperçu des données pour comprendre la structure et les valeurs manquantes éventuelles.

2) Effectuez le prétraitement des données :

- Remplacez les valeurs manquantes dans la colonne **Age** par la moyenne de la colonne.
- Encodez la colonne **Sex** avec des valeurs numériques (1 pour **male**, 0 pour **female**).
- Remplacez les valeurs manquantes dans **Cabin** par la valeur "G6".
- Encodez la colonne **Survived** en valeurs numériques (1 pour **Yes**, 0 pour **No**).

3) Sélectionnez les colonnes pertinentes pour X (caractéristiques) en excluant les colonnes non nécessaires comme **Name**, **Cabin**, **Ticket**, et **Embarked**. La colonne **Survived** sera utilisée comme cible (y).

4) Utilisez `train_test_split` de Scikit-learn pour diviser les données en ensemble d'entraînement et de test (20 % pour le test, 80 % pour l'entraînement).

5) Construire un modèle d'arbre de décision :

- Importez la classe **DecisionTreeClassifier** depuis la bibliothèque **sklearn.tree**.
- Créez un modèle DT initial sans paramètres (paramètres par défaut), puis entraînez le modèle sur l'ensemble d'entraînement avec la méthode **.fit()**.

6) Utilisez `plot_tree` de Scikit-learn pour afficher l'arbre de décision. Ensuite, répondez aux questions suivantes :

- Expliquez brièvement le graphique obtenu.
- Mentionnez si l'arbre semble trop complexe (signe possible d'overfitting) ou trop simple (sous-ajustement).
- Analysez les règles apprises et leur pertinence pour le problème.

7) Utilisez le modèle que vous avez créé pour faire des prédictions sur l'ensemble de test et de train.

Ensuite, mesurez les métriques suivantes :

- Accuracy (Exactitude)
- Precision (Précision)
- Recall (Rappel)
- F1 Score (Score F1)

Sur les deux ensembles test et train, interprétez les résultats.

8) Construisez un second modèle d'arbre de décision en utilisant `GridSearchCV` pour optimiser les paramètres :

- Définissez une grille de paramètres à tester (par exemple, `max_depth`, `min_samples_split`, `min_samples_leaf`, `criterion`, etc.).
- Utilisez `GridSearchCV` pour rechercher les meilleures combinaisons de paramètres en fonction de la performance sur l'ensemble d'entraînement.
- Affichez les meilleurs paramètres trouvés.

```
#----- GridSearchCV
from sklearn.model_selection import GridSearchCV
my_parametres = {
    'criterion': ['gini', 'entropy'],
    'splitter' : ['best', 'random'],
    'max_depth': [3, 4, 5],
    'min_samples_leaf': [1, 2, 5, 10],
    'max_leaf_nodes': [5, 10, 15],
}
my_model = DecisionTreeClassifier()
grid_search = GridSearchCV(estimator=my_model, param_grid=my_parametres, cv=10)
grid_search.fit(x_train, y_train)
print("bst paramètres:", grid_search.best_params_)
print("bst score:", grid_search.best_score_)
```

9) Reprenez les questions 5, 6, et 7 en appliquant cette fois le modèle optimisé via **GridSearchCV**.